

Recurrent Neural Networks

CSE 5526 - Intro to Neural Networks
Lecture 16

Instructor: Donald S. Williamson

 @TheASPIREGrpOSU

 theaspiregroup_osu

 The ASPIRE Group @ OSU

Agenda and Learning Outcomes

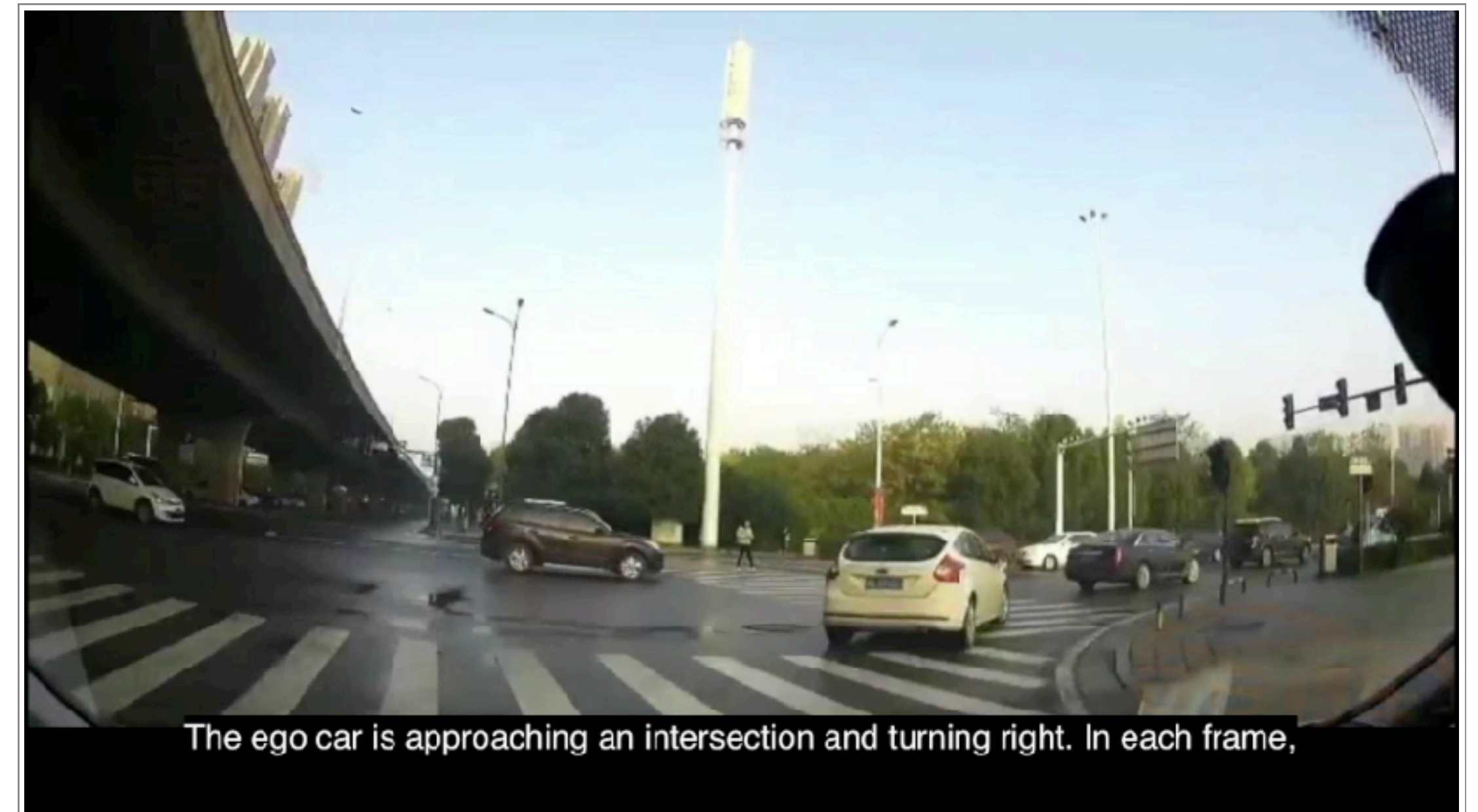
Today's Topics

- Limitations of DNNs with temporal/sequential data
- Introduce Recurrent Neural Networks (RNNs)
- Introduce the Bottom-Up graphical notation for neural networks
- Explain Backpropagation through time (BPTT) for RNNs
- **Announcements**
 - Quiz#5 has been published. Due by Friday, March 21st
 - Programming assignment #2 will be published. Due date TBD.

Example Sequential Data

Time Varying Data

- So far we have NOT formally considered data that varies over time (e.g. video, audio, health monitoring data,...) or is sequential in nature (e.g. text)
- This data will have features and labels that evolve over time (or the sequence), many of which may be correlated
 - For example,
 - Action Recognition in Videos ->
 - Live Transcription from Audio



The ego car is approaching an intersection and turning right. In each frame,

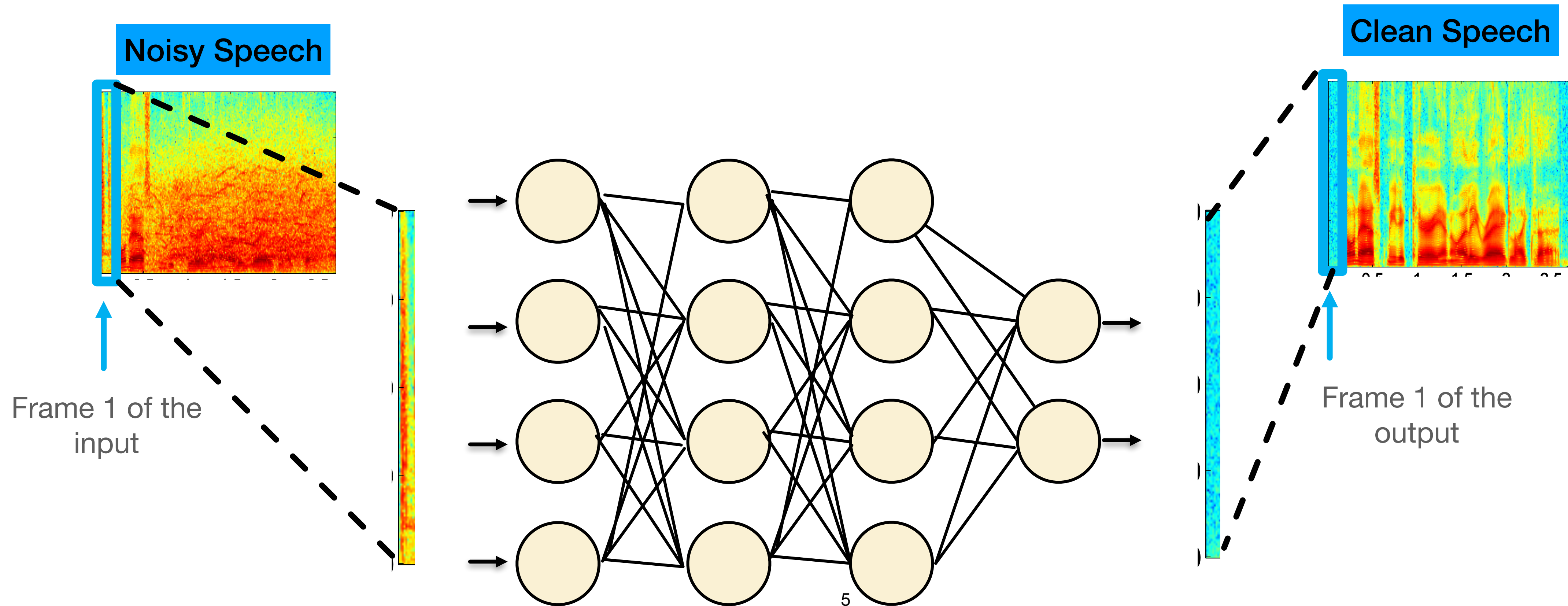
Sequential Data Examples

Think-Pair-Share

- What are examples where data is sequential and the different elements of the data sequence are related?

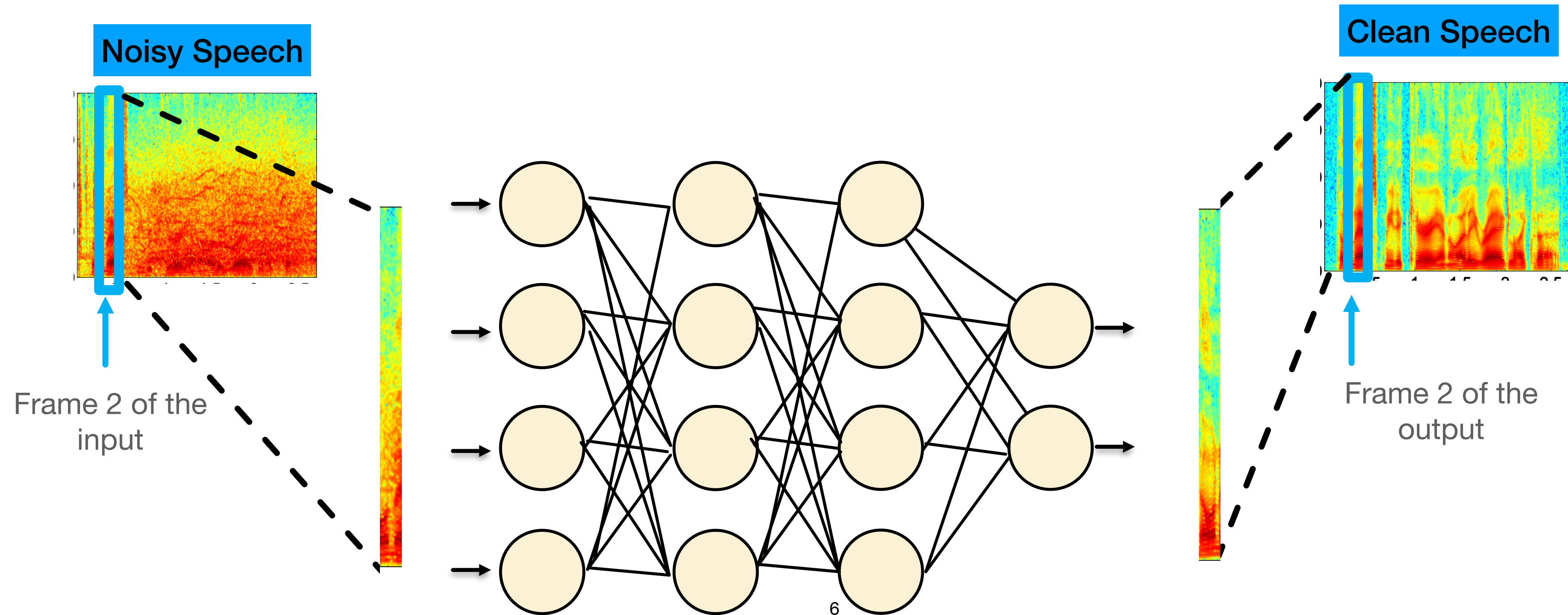
DNN for Speech Enhancement

- You work for a hearing-aid design company, and you want to use a DNN to remove unwanted background noise. This is generally done independently for small segments (or frames) of speech



DNN for Speech Enhancement

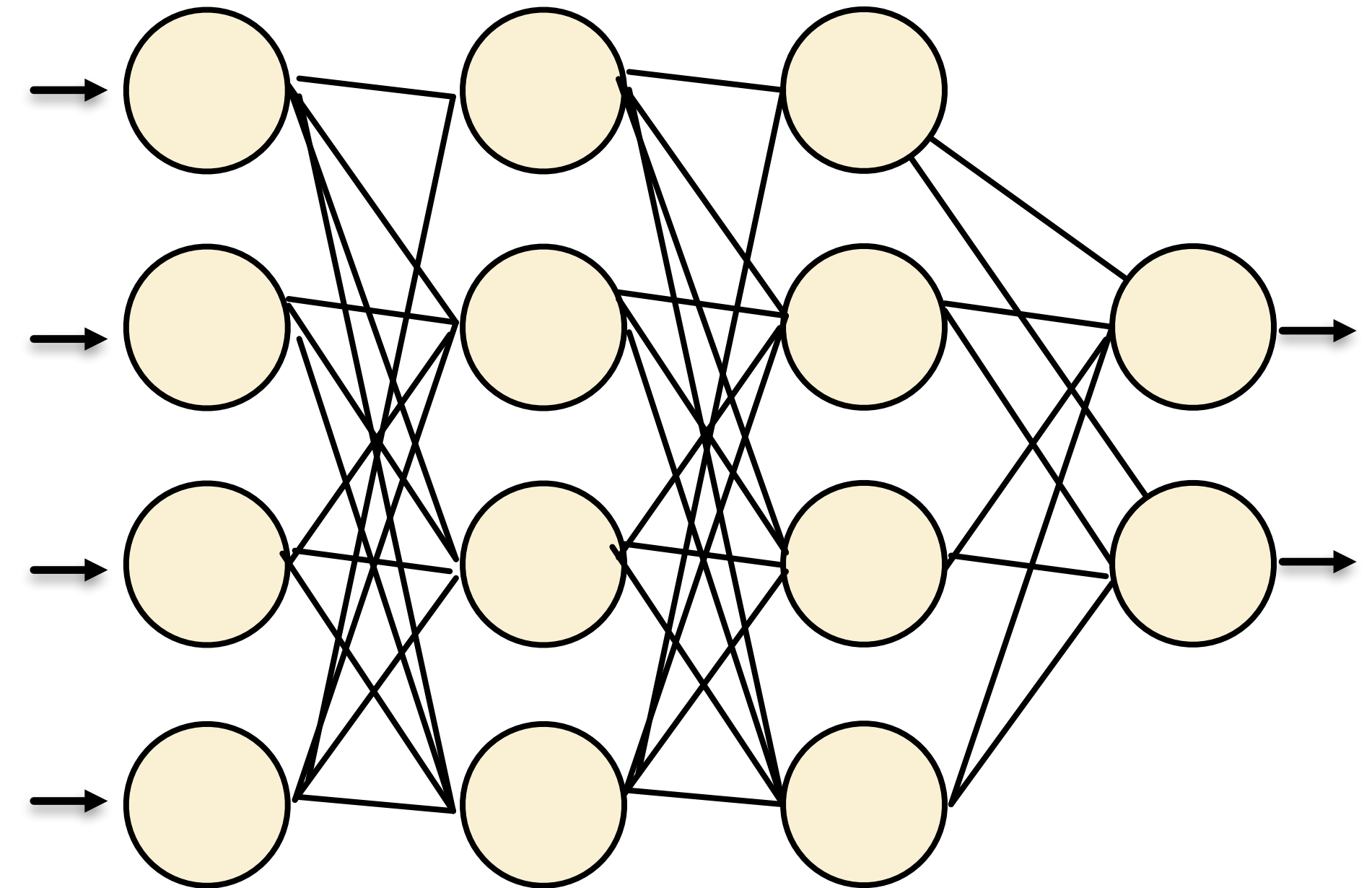
- You work for a hearing-aid design company, and you want to use a DNN to remove unwanted background noise. This is generally done independently for small segments (or frames) of speech



Erroneous Assumptions ?

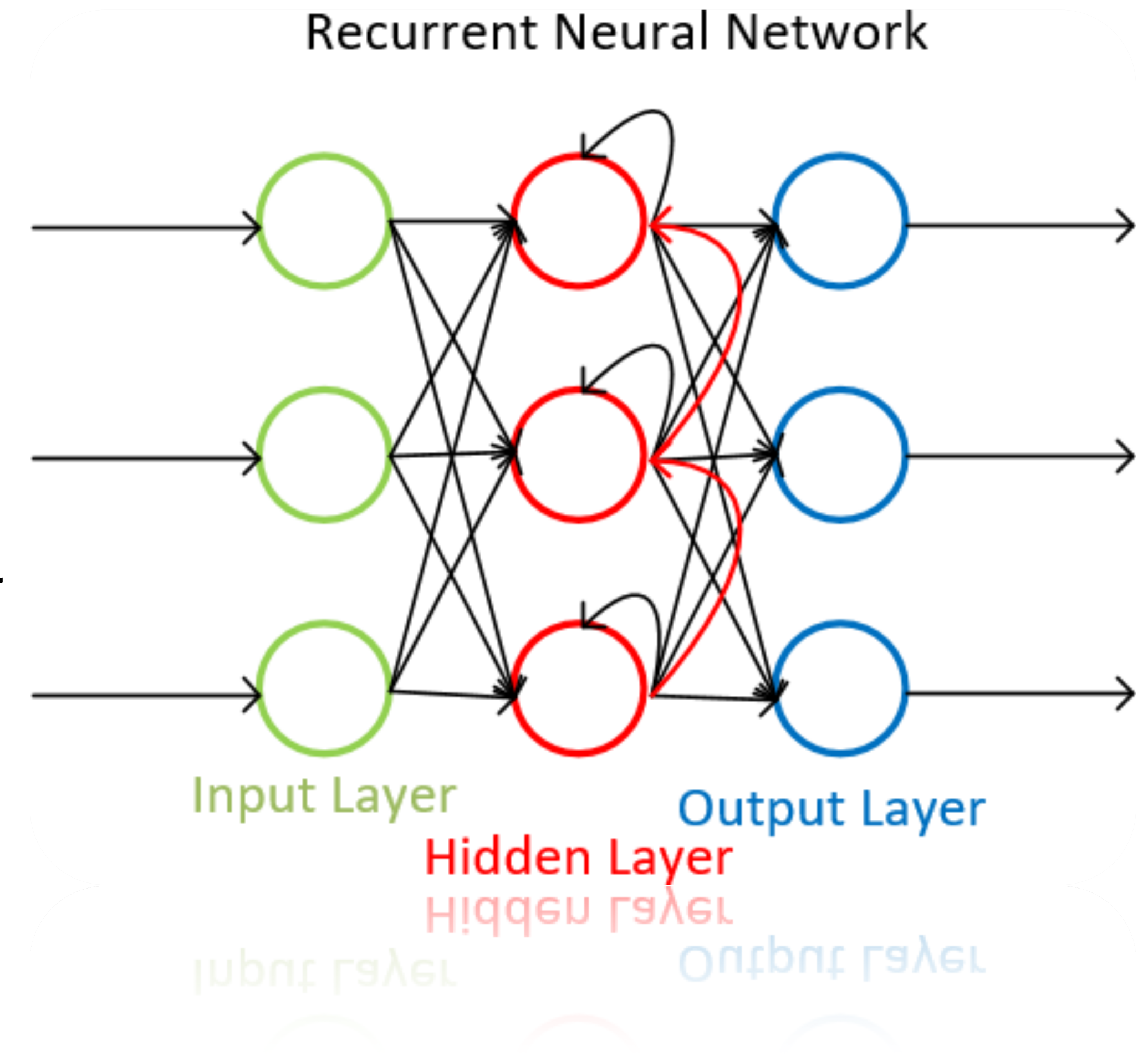
Think-Pair-Share

- What assumptions about the input and output are these models making?



Recurrent Neural Networks (RNNs)

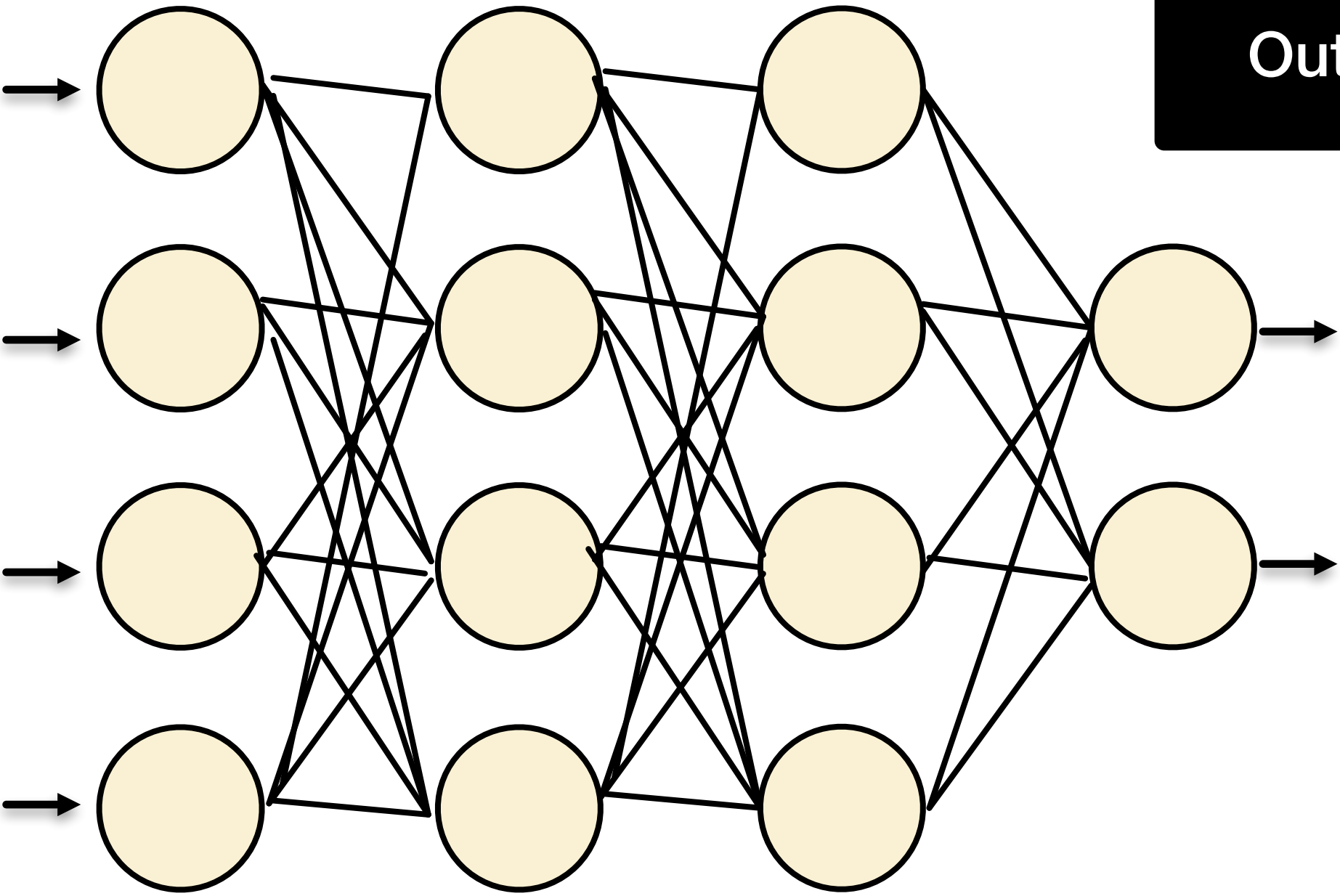
- RNNs have “memory”
 - Remember prior calculations
 - This is done through loops (or cycles)
 - Current output value is seen one-time step (or sequence) later
- RNNs are ideal for handling sequential data
 - Traditional neural networks assume that all inputs (and outputs) are independent of each other
 - This is not true for many tasks (i.e. speech recognition, natural language processing)



Network Notation

A Modified View

Feed-Forward Notation

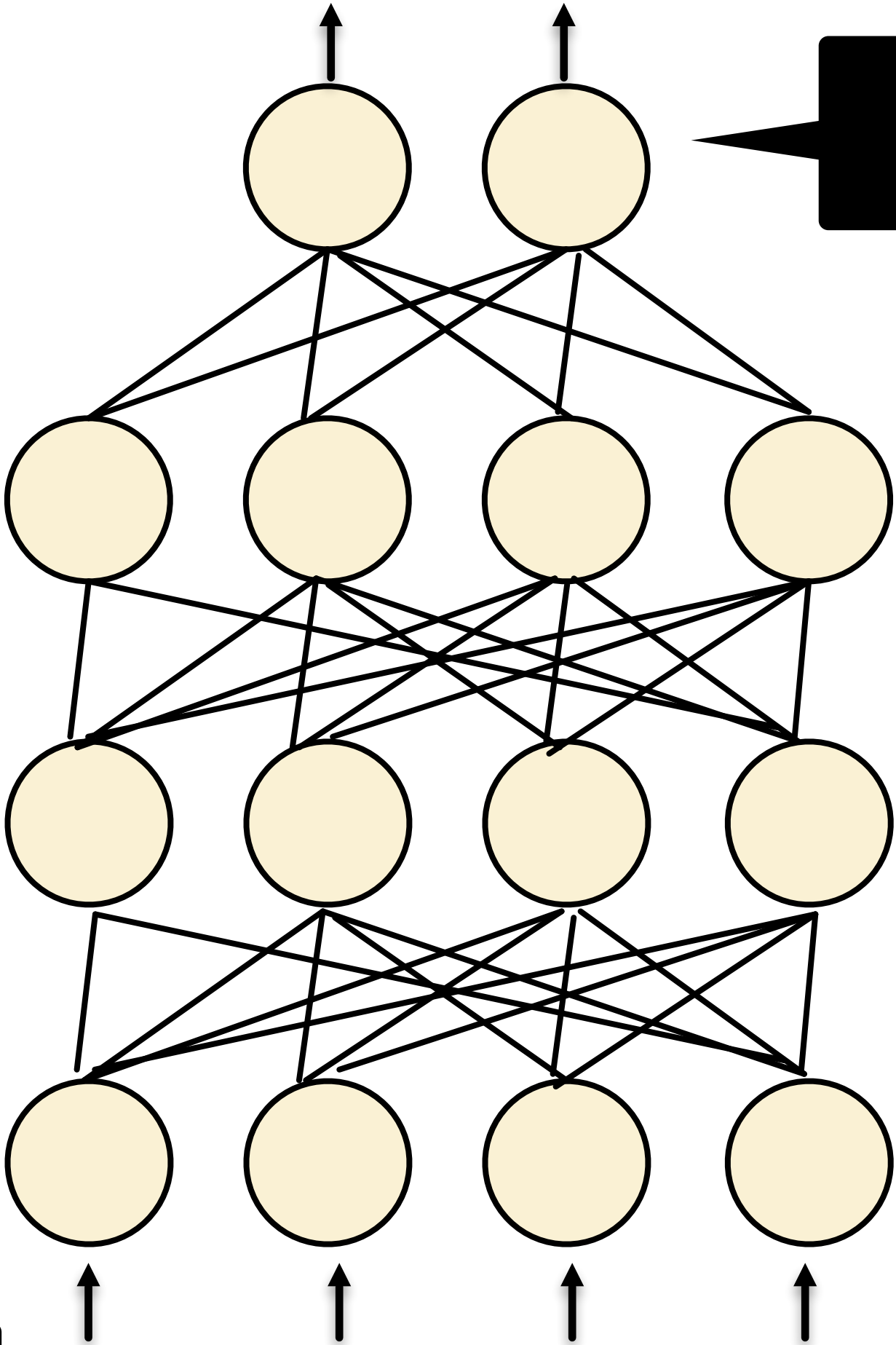


Input

Output

=>

Bottom-Up Notation

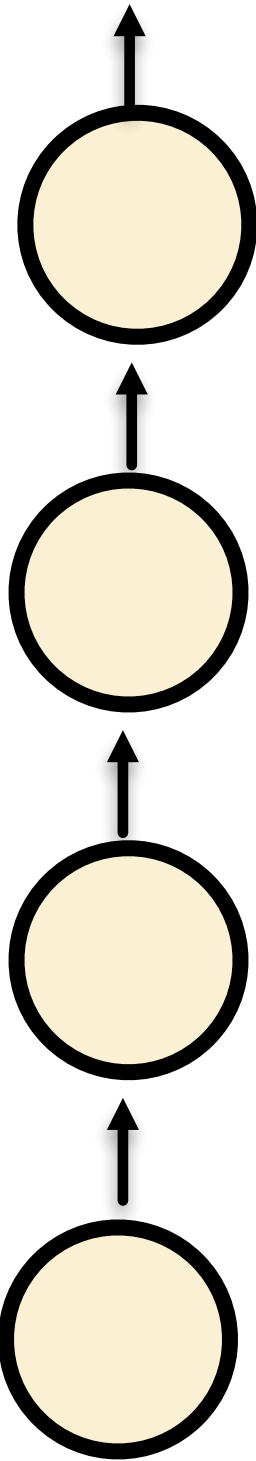


Input

Output

=>

Further Simplified

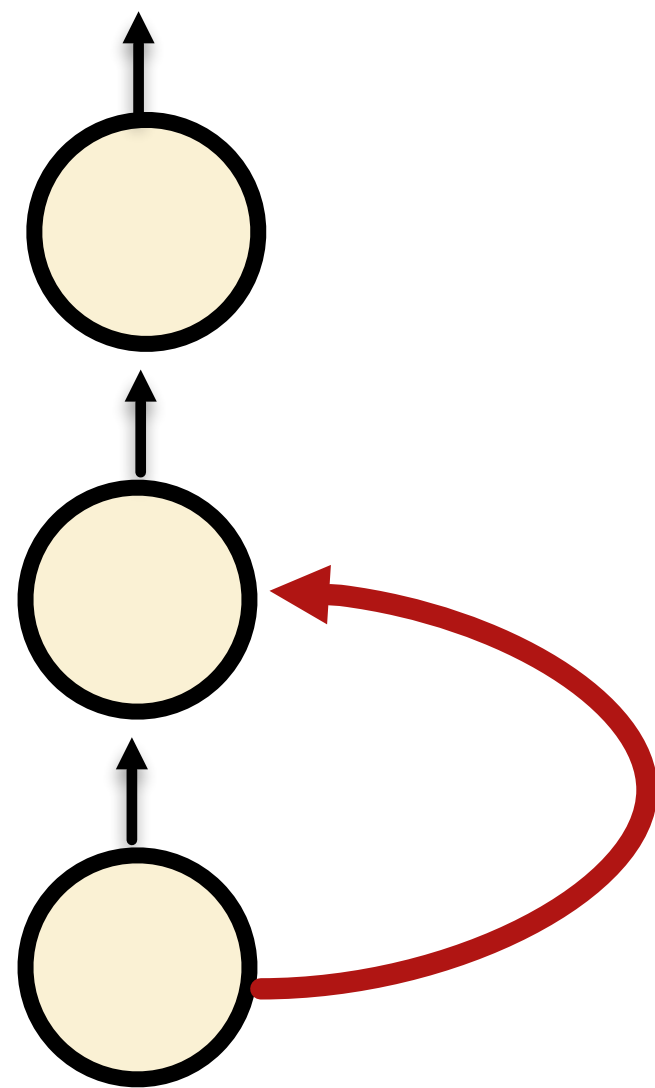


Input

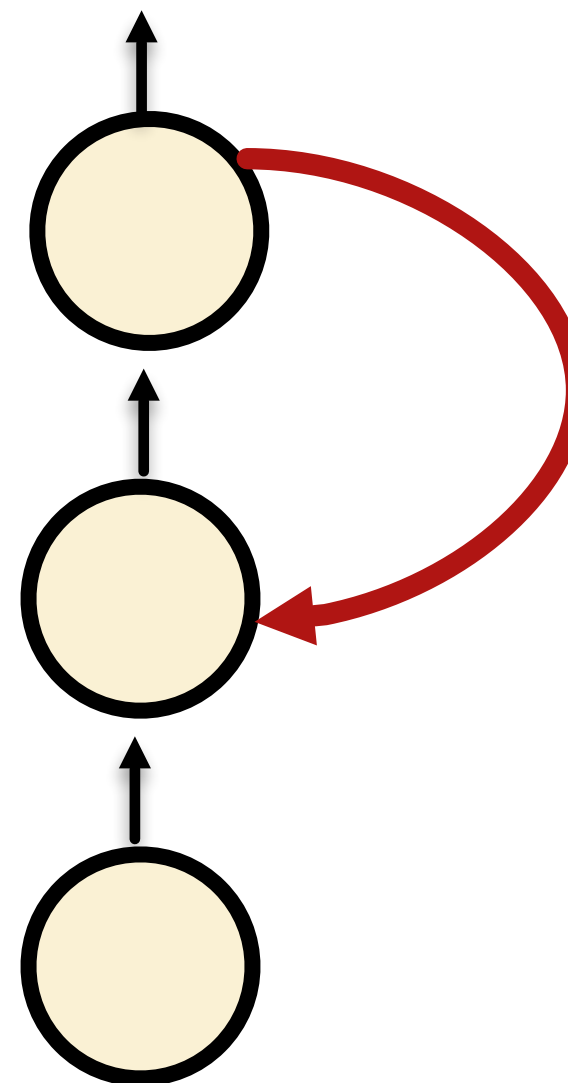
Enforcing Correlations and Dependencies in the Network

Temporal (or sequential) Correlations or Dependencies can be enforced by adding recurrent connections within the network. Recurrent connections are delayed (e.g., become inputs at a later time)

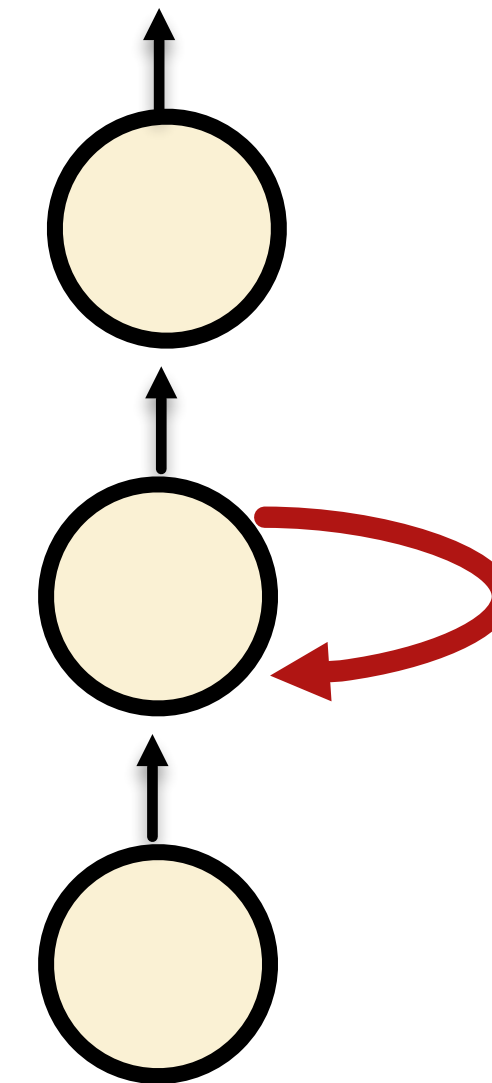
Ex. Input layer
Recurrency



Ex. Output layer
Recurrency



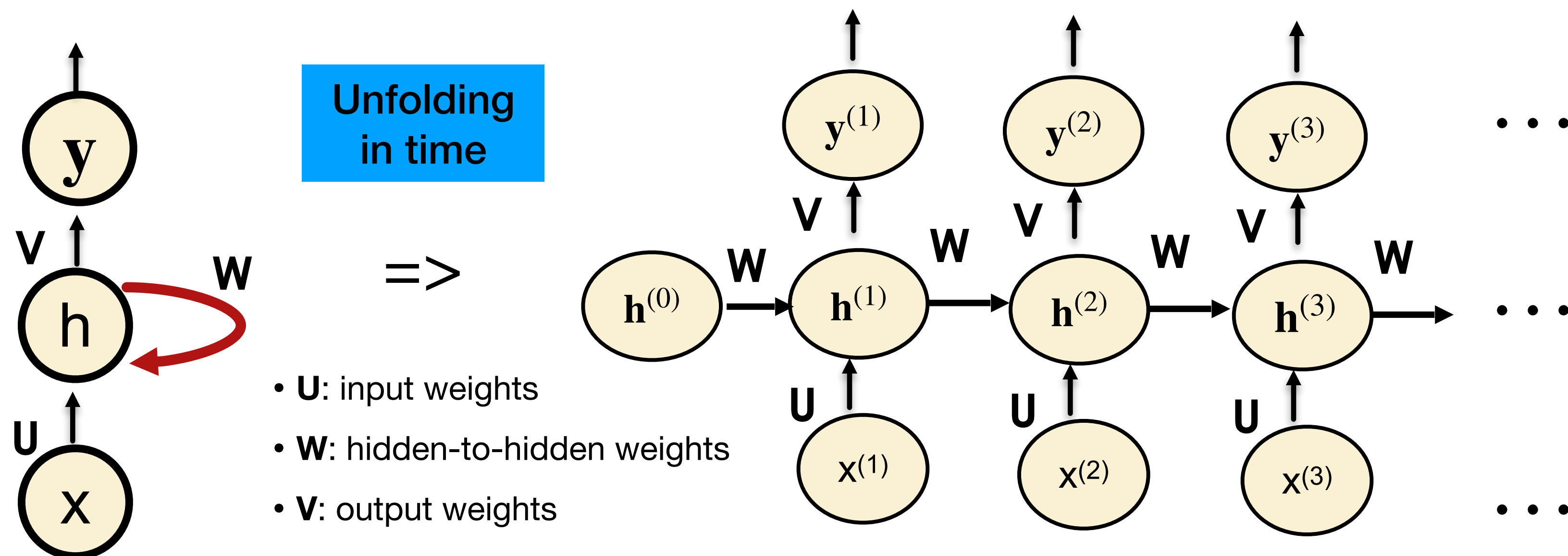
Ex. Hidden layer
Recurrency



Recurrent Hidden Units (RHU)

Forward pass of RNN with RHU: Unfold over time

- Example of sequence-to-sequence mapping
- Assume a sequence of length T (e.g. there are T separate input features and T separate corresponding labels - $x^{(t)}$ represents a frame of a video)



- $\mathbf{U}$: input weights
- $\mathbf{W}$: hidden-to-hidden weights
- $\mathbf{V}$: output weights
- $\mathbf{y}$: network output
- $\mathbf{h}$: hidden layer output
- $\mathbf{x}$: network input

$$\mathbf{v}^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)}$$

$$\mathbf{h}^{(t)} = \phi(\mathbf{v}^{(t)})$$

$$\mathbf{y}^{(t)} = \phi(\mathbf{V}\mathbf{h}^{(t)} + \mathbf{c})$$

$\mathbf{b}$ and $\mathbf{c}$ are bias terms
 $\mathbf{h}^{(0)}$ is initialized to 0

Questions

RHU Error Calculation

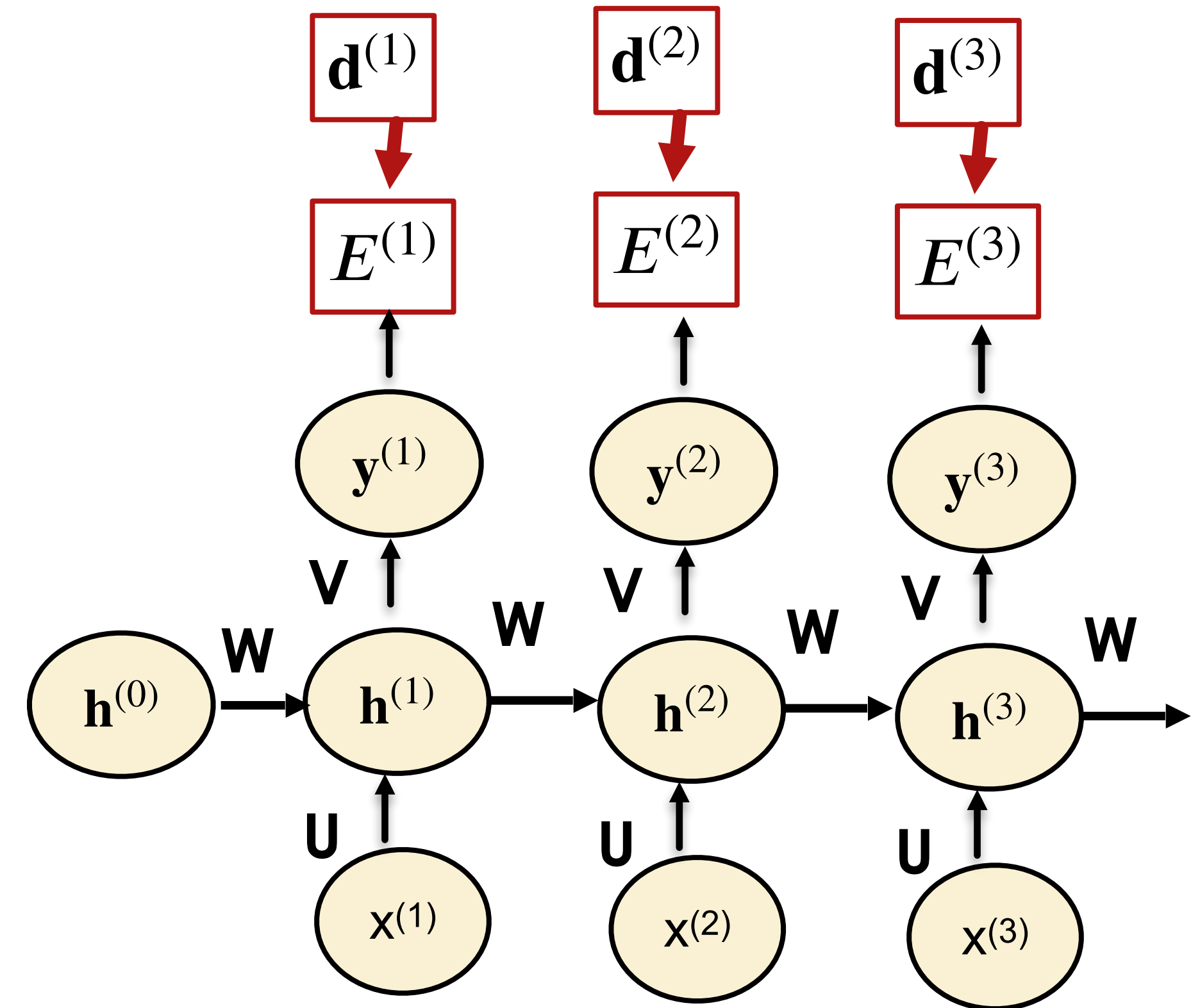
- Assume a square error loss function, with $\mathbf{d}^{(t)}$ the desired output at time t
- Element-wise error (loss function)**

$$E^{(t)}(y^{(t)}, d^{(t)}) = \frac{1}{2}(d^{(t)} - y^{(t)})^2$$

- **Sequence Error (e.g. individual video)**

$$E_s = \sum_{t=1}^T E^{(t)}(y^{(t)}, d^{(t)})$$

Total error is calculated across all sequence outputs (e.g. all videos, not just one)

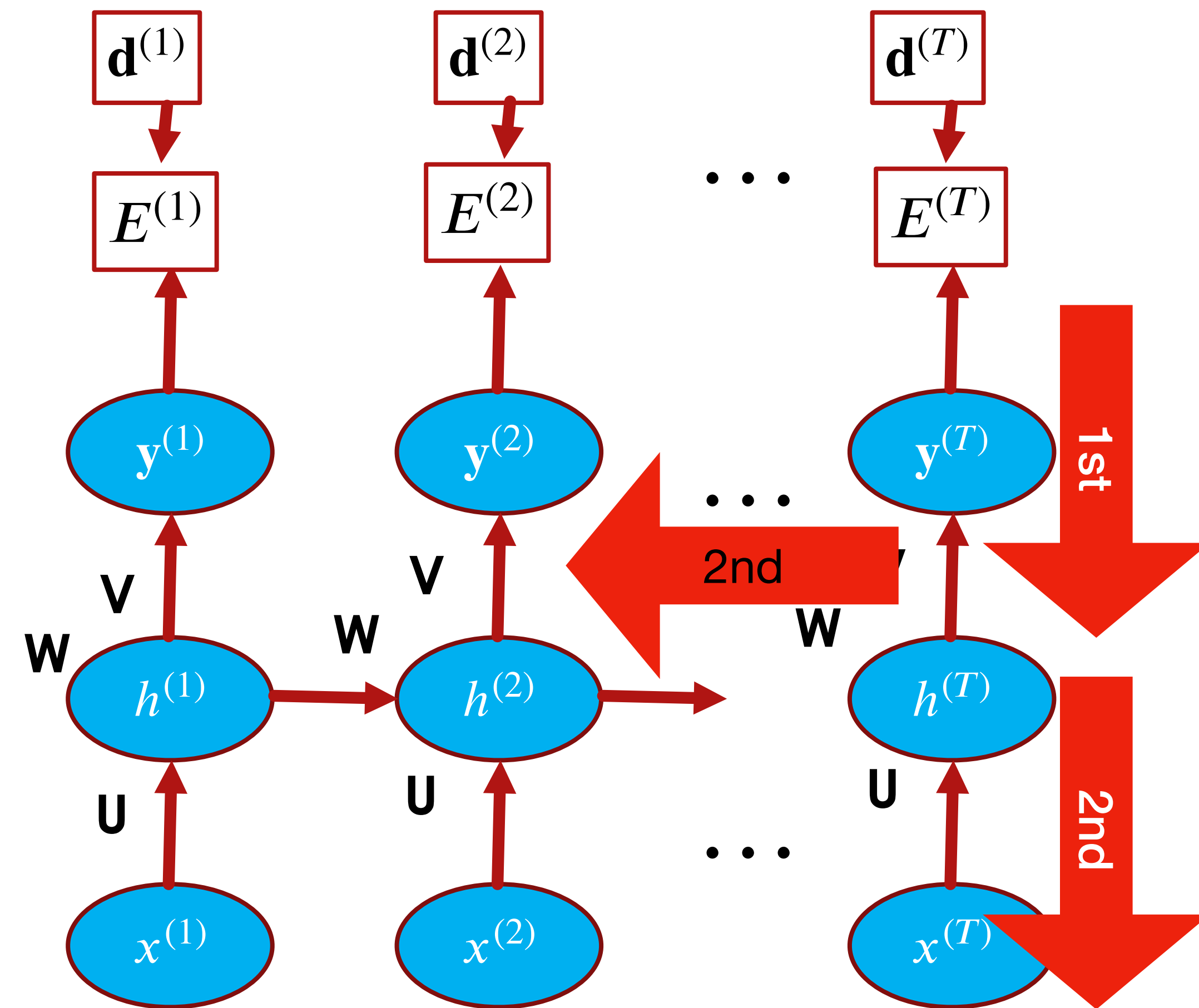


- **Total Data Error (across all sequences)**

$$E_{total} = \sum_{s=1}^S E_s$$

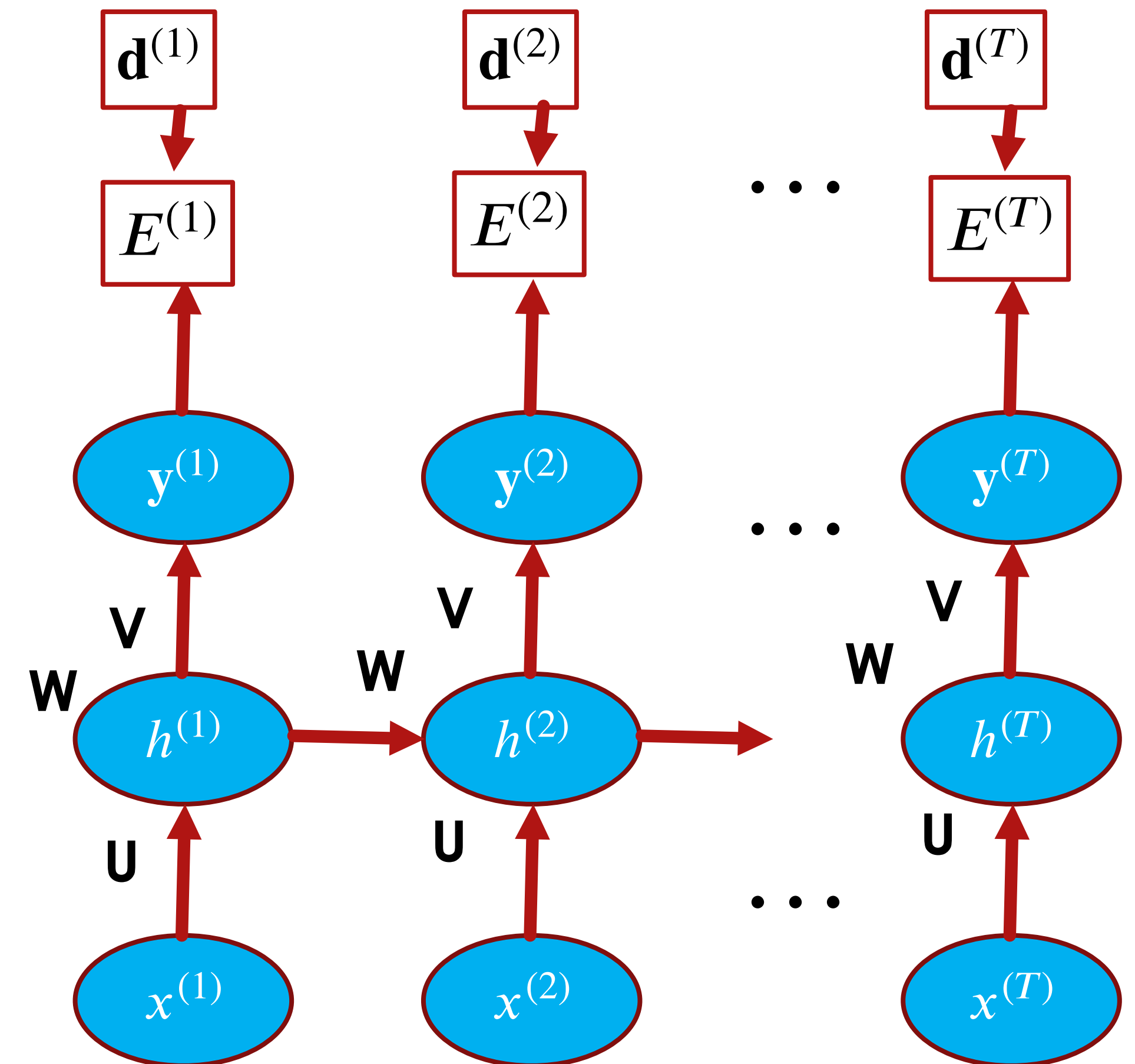
Backpropagation through Time (BPTT)

- Treat the unfolded RNN as a DNN (e.g. feed-forward network with multiple hidden layers, and multiple output layers)
- Weights, however, are the same for each “layer”, unlike a DNN.
- Propagate the error backwards as before, but now it is also through time



Gradient Calculations for RNN with RHU

- Need to learn weight (e.g., $\mathbf{U}$, $\mathbf{W}$, $\mathbf{V}$) and bias terms (e.g., $\mathbf{b}$, $\mathbf{c}$) for the RNN. Biases can be incorporated into the weight vectors
- Three gradients need to be computed. One for the:
 - Output weights: $\nabla_{\mathbf{V}} E$
 - Hidden-to-hidden weights: $\nabla_{\mathbf{W}} E$
 - Input weights: $\nabla_{\mathbf{U}} E$



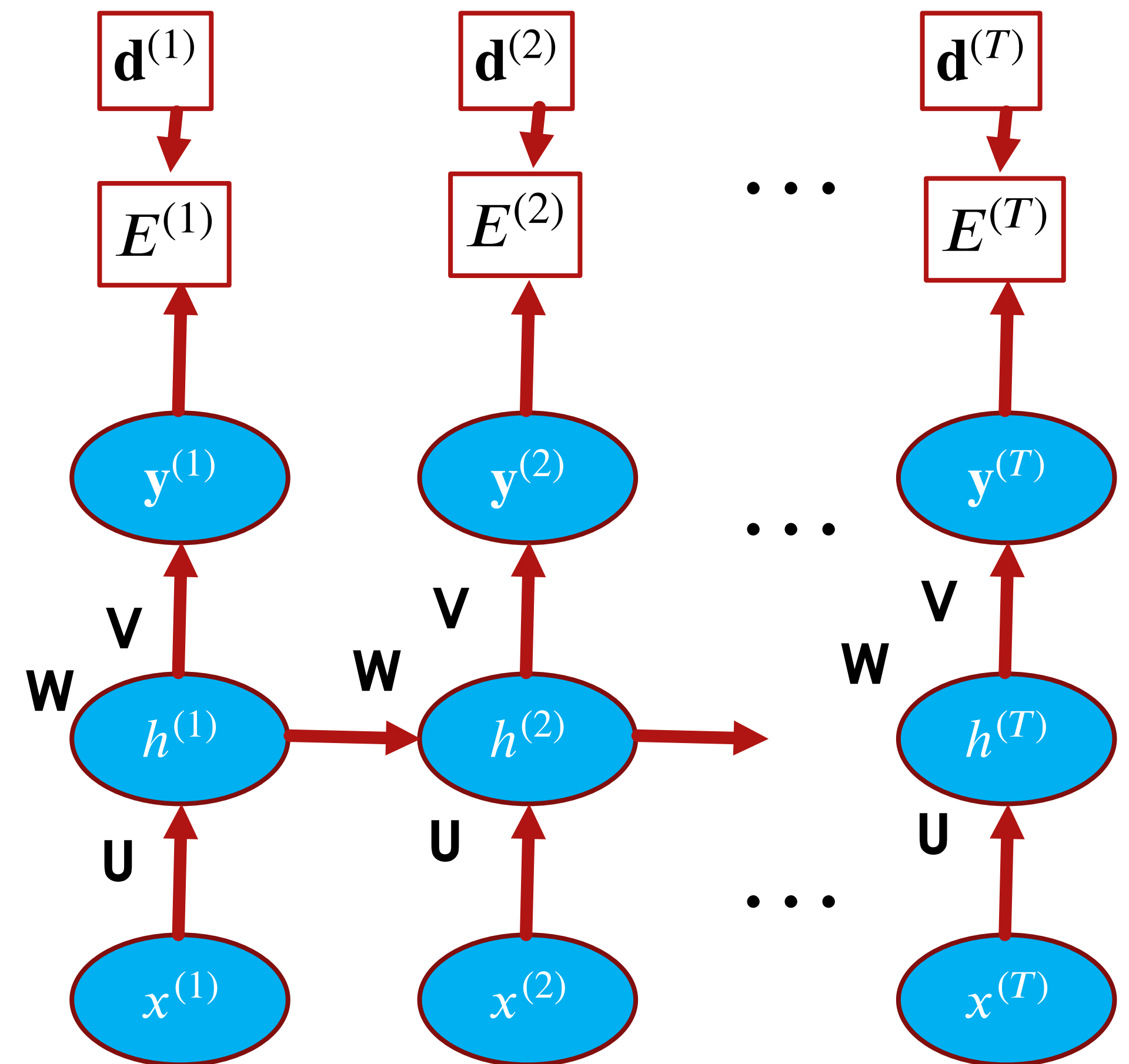
Gradient Calculations for RNN with RHU

- We then can utilize gradient descent to update the weights

$$\mathbf{V}(n+1) = \mathbf{V}(n) - \eta_{\mathbf{V}} \nabla_{\mathbf{V}} E(n)$$

$$\mathbf{W}(n+1) = \mathbf{W}(n) - \eta_{\mathbf{W}} \nabla_{\mathbf{W}} E(n)$$

$$\mathbf{U}(n+1) = \mathbf{U}(n) - \eta_{\mathbf{U}} \nabla_{\mathbf{U}} E(n)$$



BPTT: Gradient for the Output Weights, $\mathbf{V}$

- **Weight update rule:** $\mathbf{V}(n + 1) = \mathbf{V}(n) - \eta_{\mathbf{V}} \nabla_{\mathbf{V}} E(n)$

- **Sequence Error:**

$$\begin{aligned} E_s &= \sum_{t=1}^T E^{(t)}(y^{(t)}, d^{(t)}) = \sum_{t=1}^T \frac{1}{2} (d^{(t)} - y^{(t)})^2 = \sum_{t=1}^T \frac{1}{2} (d^{(t)} - \phi(p^{(t)}))^2 \\ &= \sum_{t=1}^T \frac{1}{2} (d^{(t)} - \phi(\mathbf{V}\mathbf{h}^{(t)} + c))^2 \end{aligned}$$

- **Hence,**

$$\nabla_{\mathbf{V}} E = \sum_{t=1}^T \frac{\partial E^{(t)}}{\partial \mathbf{V}} = \sum_{t=1}^T \frac{\partial E^{(t)}}{\partial y^{(t)}} \frac{\partial y^{(t)}}{\partial p^{(t)}} \frac{\partial p^{(t)}}{\partial \mathbf{V}} = - \sum_{t=1}^T (d^{(t)} - y^{(t)}) \phi'(p^{(t)}) \mathbf{h}^{(t)}$$

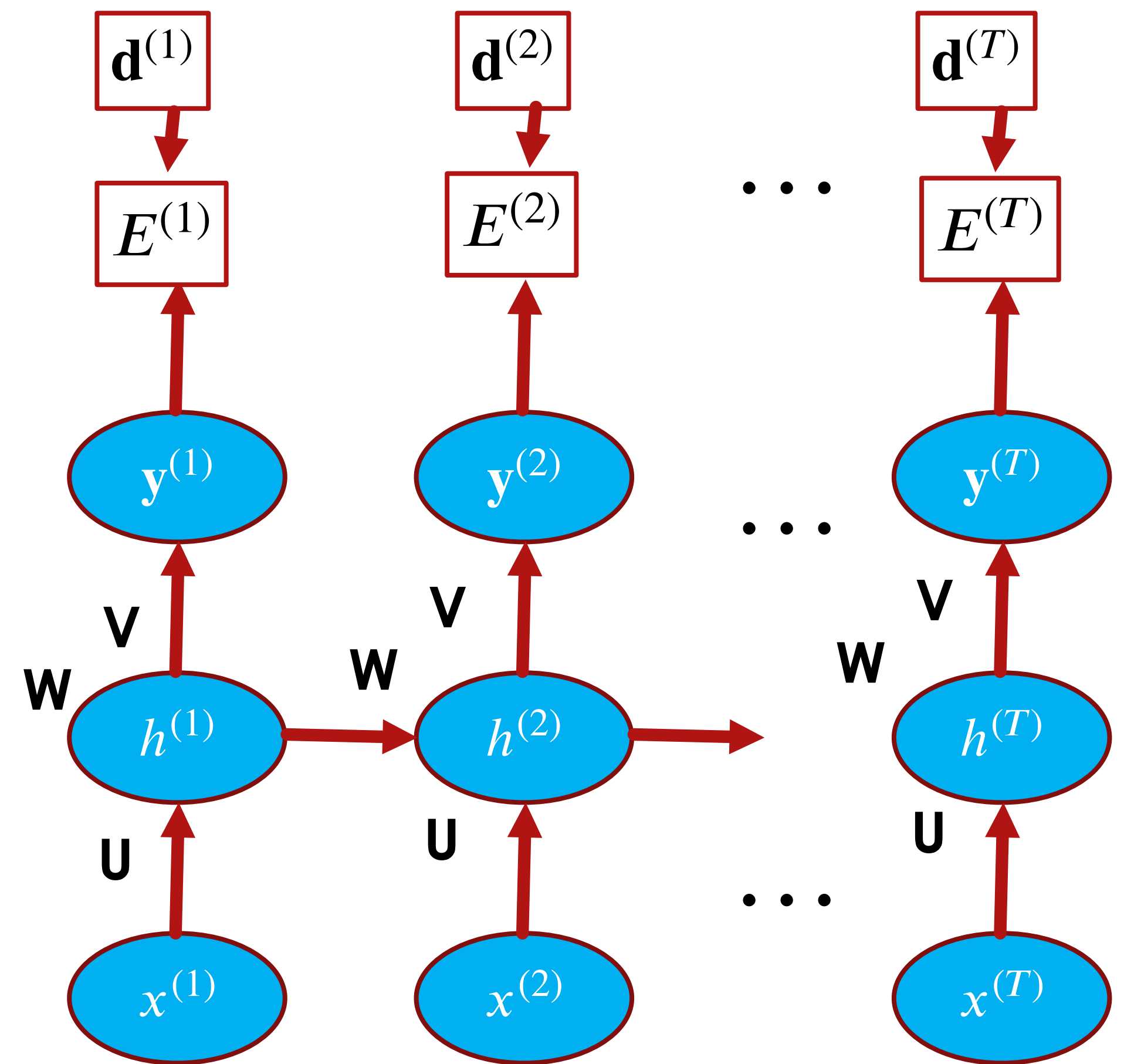
BPTT: Gradient for the Output Weights, V

- Gradient for the output weights,

$$\begin{aligned}\nabla_{\mathbf{V}} E &= - \sum_{t=1}^T (d^{(t)} - y^{(t)}) \phi'(p^{(t)}) \mathbf{h}^{(t)} \\ &= - \sum_{t=1}^T \delta_y^{(t)} \mathbf{h}^{(t)}\end{aligned}$$

- The weight update for the output weights becomes:

$$\mathbf{V}(n+1) = \mathbf{V}(n) + \eta_{\mathbf{V}} \sum_{t=1}^T \delta_y^{(t)} \mathbf{h}^{(t)}$$



BPTT: Gradient for the Recurrent Weights, $\mathbf{W}$

- **Weight update rule:** $\mathbf{W}(n + 1) = \mathbf{W}(n) - \eta_{\mathbf{W}} \nabla_{\mathbf{W}} E(n)$

- **Sequence Error:**

$$E_s = \sum_{t=1}^T E^{(t)}(y^{(t)}, d^{(t)}) = \sum_{t=1}^T \frac{1}{2} (d^{(t)} - y^{(t)})^2 = \sum_{t=1}^T \frac{1}{2} (d^{(t)} - \phi(p^{(t)}))^2$$

- **Hence,**

$$= \sum_{t=1}^T \frac{1}{2} (d^{(t)} - \phi(\mathbf{V}\mathbf{h}^{(t)} + c))^2$$

$$\nabla_{\mathbf{W}} E = \sum_{t=1}^T \frac{\partial E^{(t)}}{\partial \mathbf{W}}$$

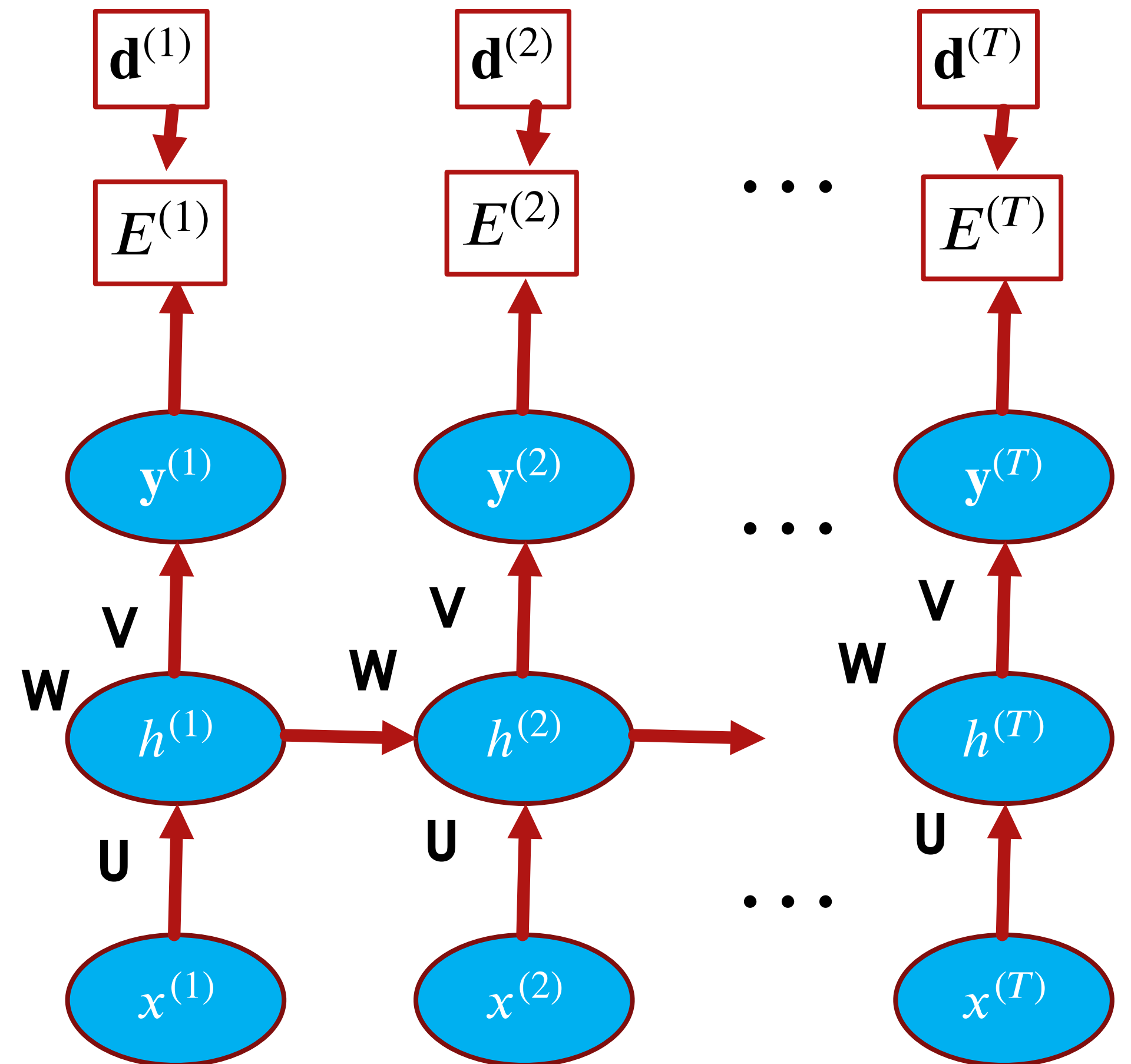
Gradient for Recurrent Weights, W

- Continuing with the hidden unit's computation

$$\mathbf{y}^{(t)} = \phi(p^{(t)}) = \phi(\mathbf{V}\mathbf{h}^{(t)} + c)$$

$$\mathbf{h}^{(t)} = \phi(v^{(t)})$$

$$v^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + b + \mathbf{W}\mathbf{h}^{(t-1)}$$



Gradient for Recurrent Weights, $\mathbf{W}$

At time T , i.e., the end of the sequence

$$\mathbf{y}^{(t)} = \phi(p^{(t)}) = \phi(\mathbf{V}\mathbf{h}^{(t)} + c)$$

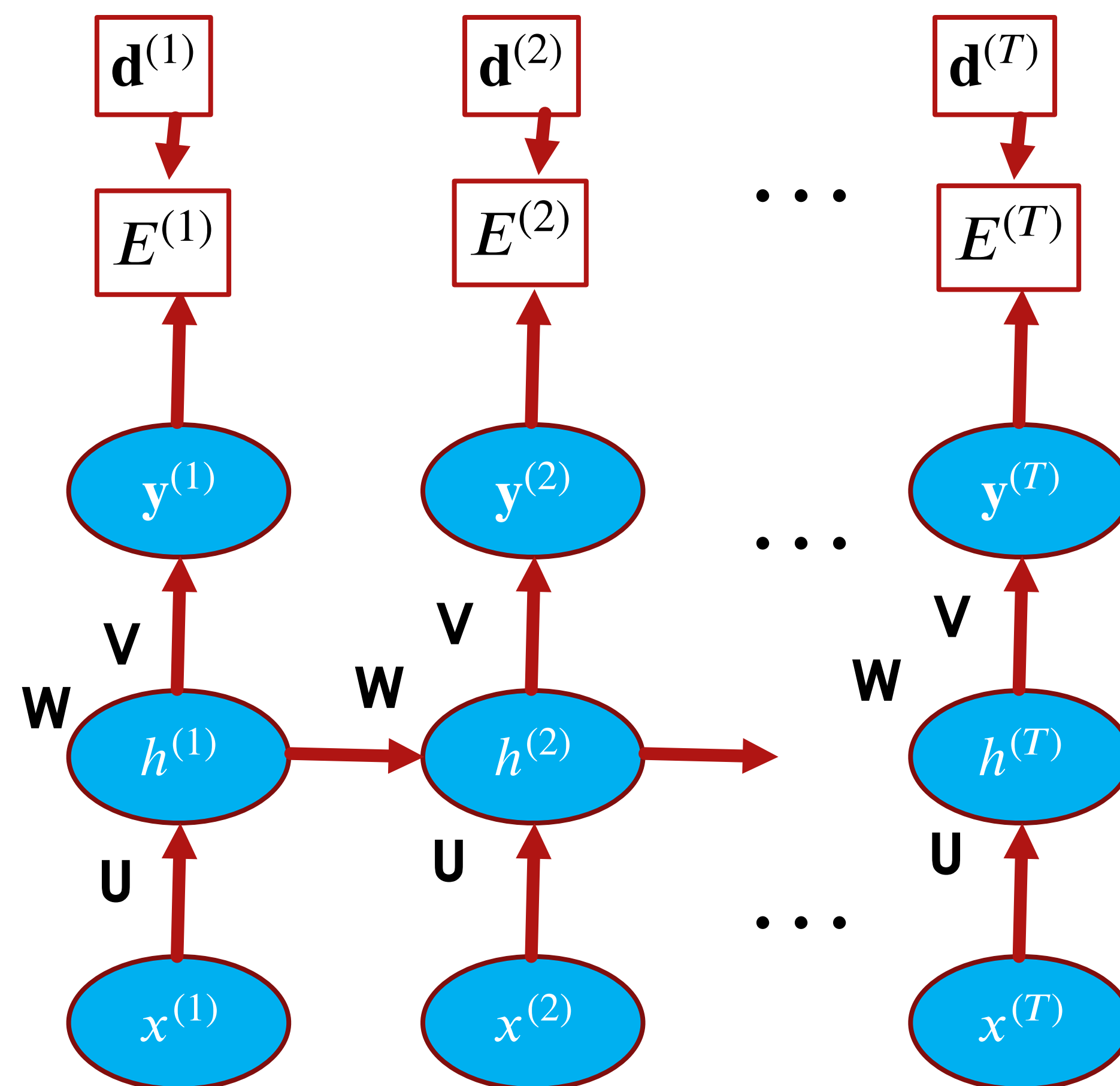
$$\mathbf{h}^{(t)} = \phi(v^{(t)})$$

$$v^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + b + \mathbf{W}\mathbf{h}^{(t-1)}$$

- Let's compute the gradient at the last element of the sequence (e.g., at time T)

$$E^{(T)} = \frac{1}{2}(d^{(T)} - y^{(T)})^2$$

$$\begin{aligned} \frac{\partial E^{(T)}}{\partial \mathbf{W}} &= \frac{\partial E^{(T)}}{\partial y^{(T)}} \frac{\partial y^{(T)}}{\partial p^{(T)}} \frac{\partial p^{(T)}}{\partial \mathbf{h}^{(T)}} \frac{\partial \mathbf{h}^{(T)}}{\partial v^{(T)}} \frac{\partial v^{(T)}}{\partial \mathbf{W}} \\ &= - (d^{(T)} - y^{(T)}) \phi'(p^{(T)}) \mathbf{V} \phi'(v^{(T)}) \mathbf{h}^{(T-1)} \\ &= - \delta_y^{(T)} \mathbf{V} \phi'(v^{(T)}) \mathbf{h}^{(T-1)} = - \delta_h^{(T)} \mathbf{h}^{(T-1)} \end{aligned}$$



Gradient for Recurrent Weights, $\mathbf{W}$

From $t = 1$ to $T-1$

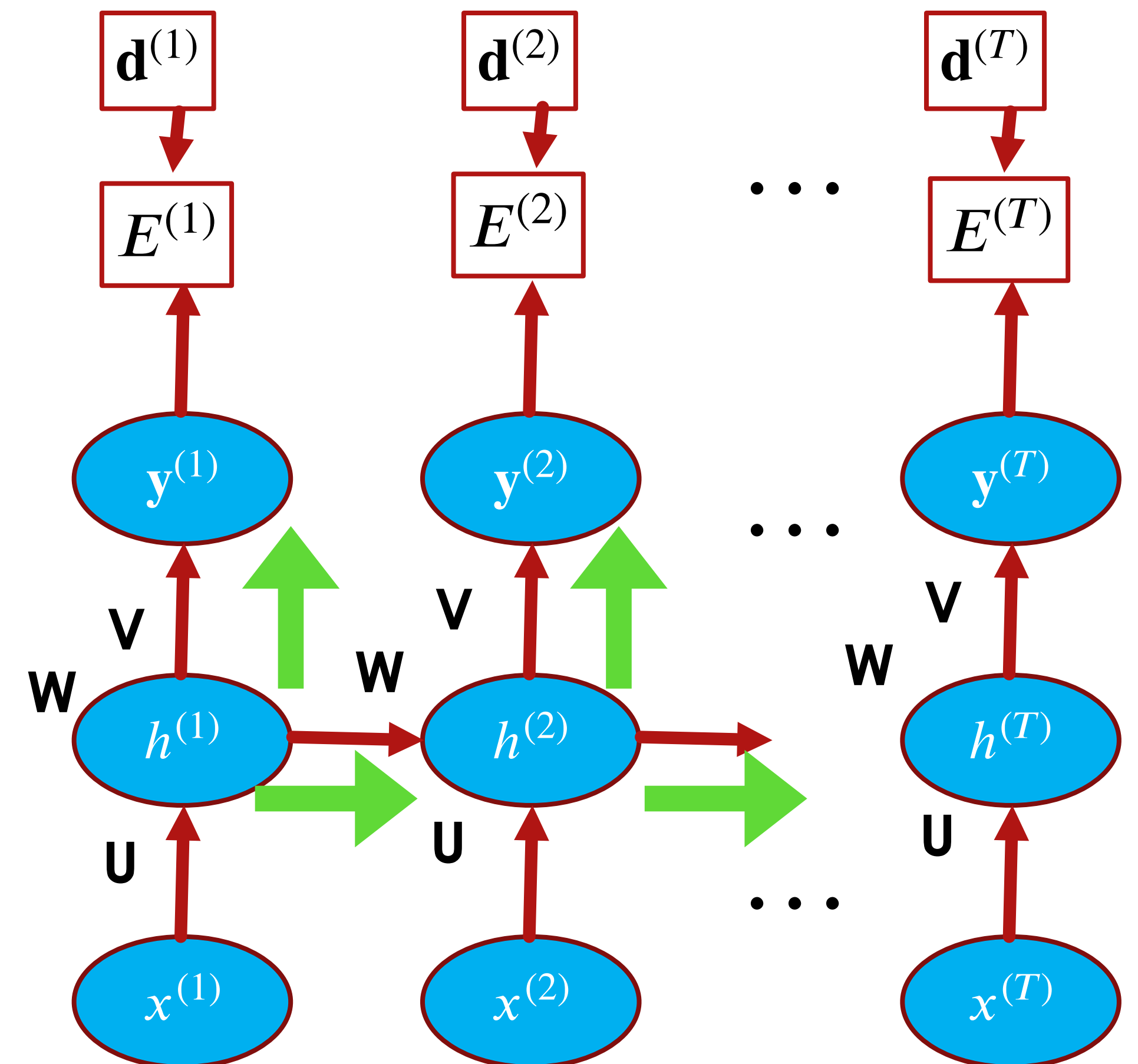
$$\mathbf{y}^{(t)} = \phi(p^{(t)}) = \phi(\mathbf{V}\mathbf{h}^{(t)} + c)$$

$$\mathbf{h}^{(t)} = \phi(v^{(t)})$$

$$v^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + b + \mathbf{W}\mathbf{h}^{(t-1)}$$

- For all but the last element of the sequence (e.g., for $t = 1$ through $T-1$), the hidden state, $h^{(t)}$, has two forward paths
 - Towards the output, $y^{(t)}$
 - Towards future hidden units, $h^{(t+1)}$
- This causes the gradient at time t , to have two additive components

$$\frac{\partial E^{(t)}}{\partial \mathbf{W}} = \frac{\partial E_{for}^{(t+1)}}{\partial \mathbf{W}} + \frac{\partial E_{out}^{(t)}}{\partial \mathbf{W}}$$



Gradient for Recurrent Weights, $\mathbf{W}$

From $t = 1$ to $T-1$

$$\mathbf{y}^{(t)} = \phi(p^{(t)}) = \phi(\mathbf{V}\mathbf{h}^{(t)} + c)$$

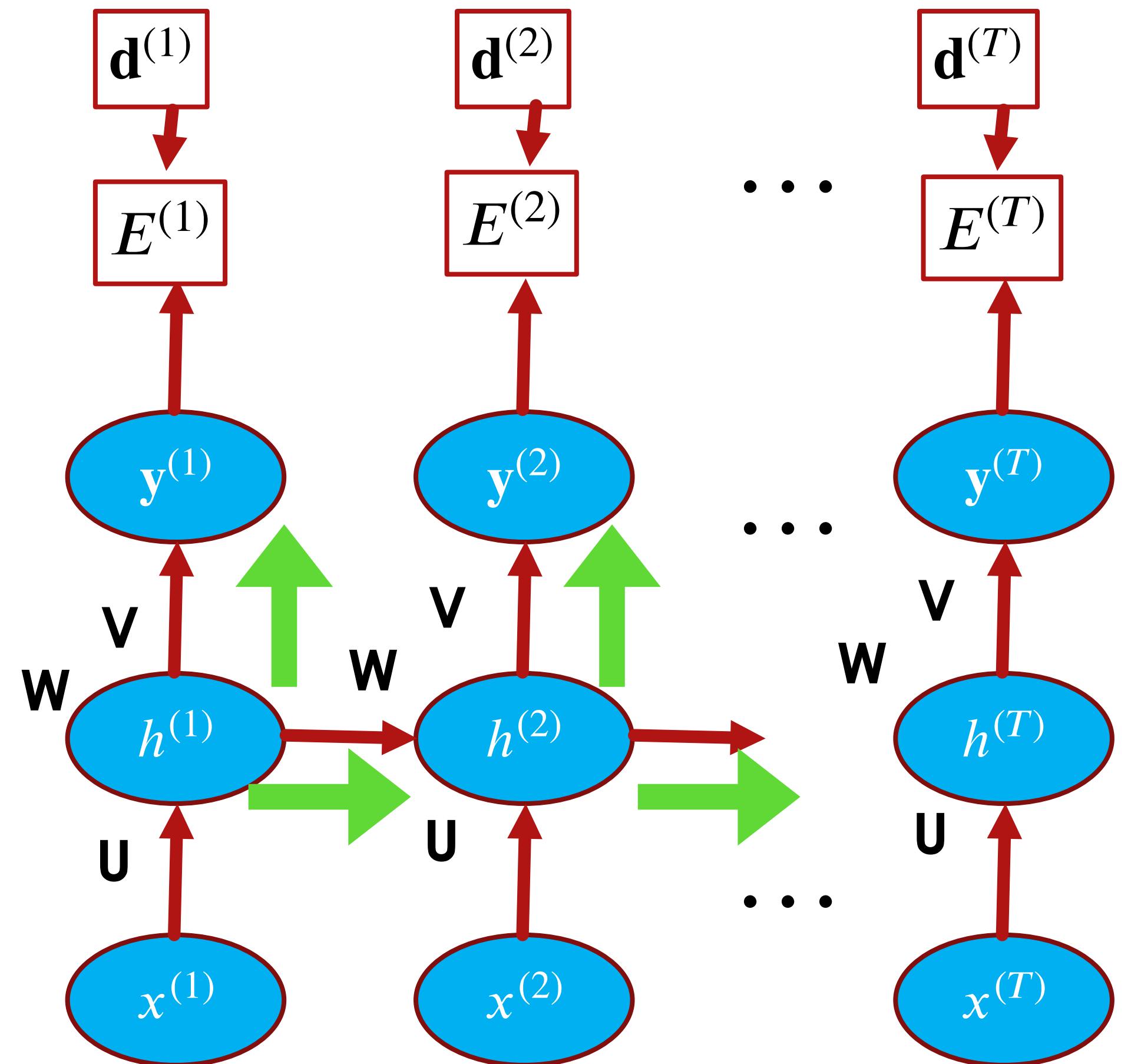
$$\mathbf{h}^{(t)} = \phi(v^{(t)})$$

$$v^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + b + \mathbf{W}\mathbf{h}^{(t-1)}$$

- This causes the gradient at time t , to have two additive components

$$\begin{aligned} \frac{\partial E^{(t)}}{\partial \mathbf{W}} &= \frac{\partial E_{for}^{(t+1)}}{\partial \mathbf{W}} + \frac{\partial E_{out}^{(t)}}{\partial \mathbf{W}} \\ &= \frac{\partial E^{(t+1)}}{\partial v^{(t+1)}} \frac{\partial v^{(t+1)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial v^{(t)}} \frac{\partial v^{(t)}}{\partial \mathbf{W}} \\ &\quad + \frac{\partial E^{(t)}}{\partial p^{(t)}} \frac{\partial p^{(t)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial v^{(t)}} \frac{\partial v^{(t)}}{\partial \mathbf{W}} \end{aligned}$$

$$\frac{\partial E^{(t)}}{\partial \mathbf{W}} = - \underbrace{[\delta_h^{(t+1)} \mathbf{W} + \delta_o^{(t)} \mathbf{V}] \phi'(v^{(t)}) \mathbf{h}^{(t-1)}}_{\delta_h^{(t)}}$$



Weight Update with RHU

Need to compute gradient with respect to each weight

- For a single-hidden layer RNN

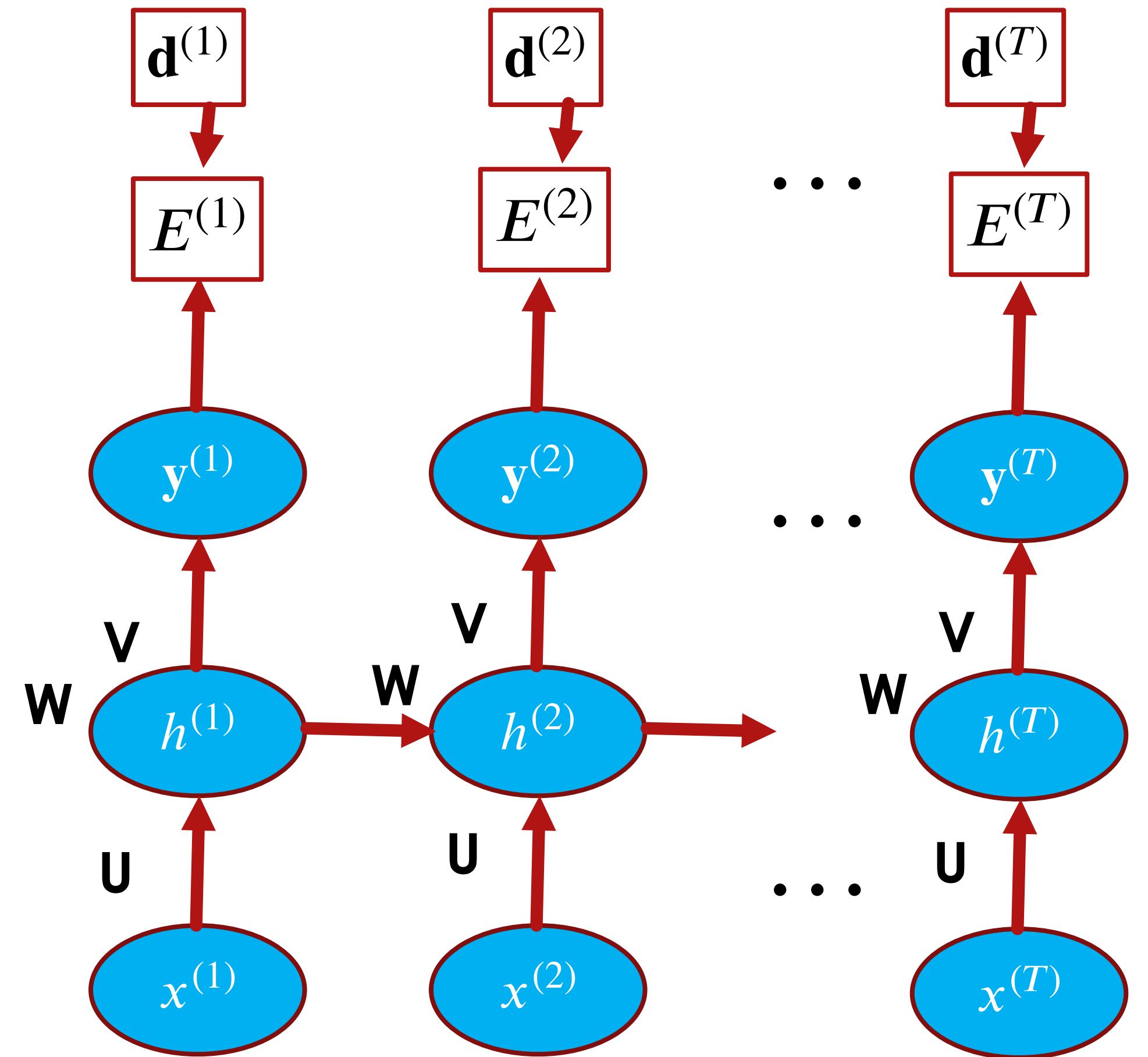
- Output layer:

$$V(n+1) = V(n) + \eta_V \sum_{t=1}^T \delta_y^{(t)} \mathbf{h}^{(t)}$$

- Hidden Layer:

$$W(n+1) = W(n) + \eta_W \sum_{t=1}^T \delta_h^{(t)} \mathbf{h}^{(t-1)}$$

- Input Layer: $U(n+1) = U(n) + \eta_U \sum_{t=1}^T \delta_h^{(t)} \mathbf{x}^{(t)}$

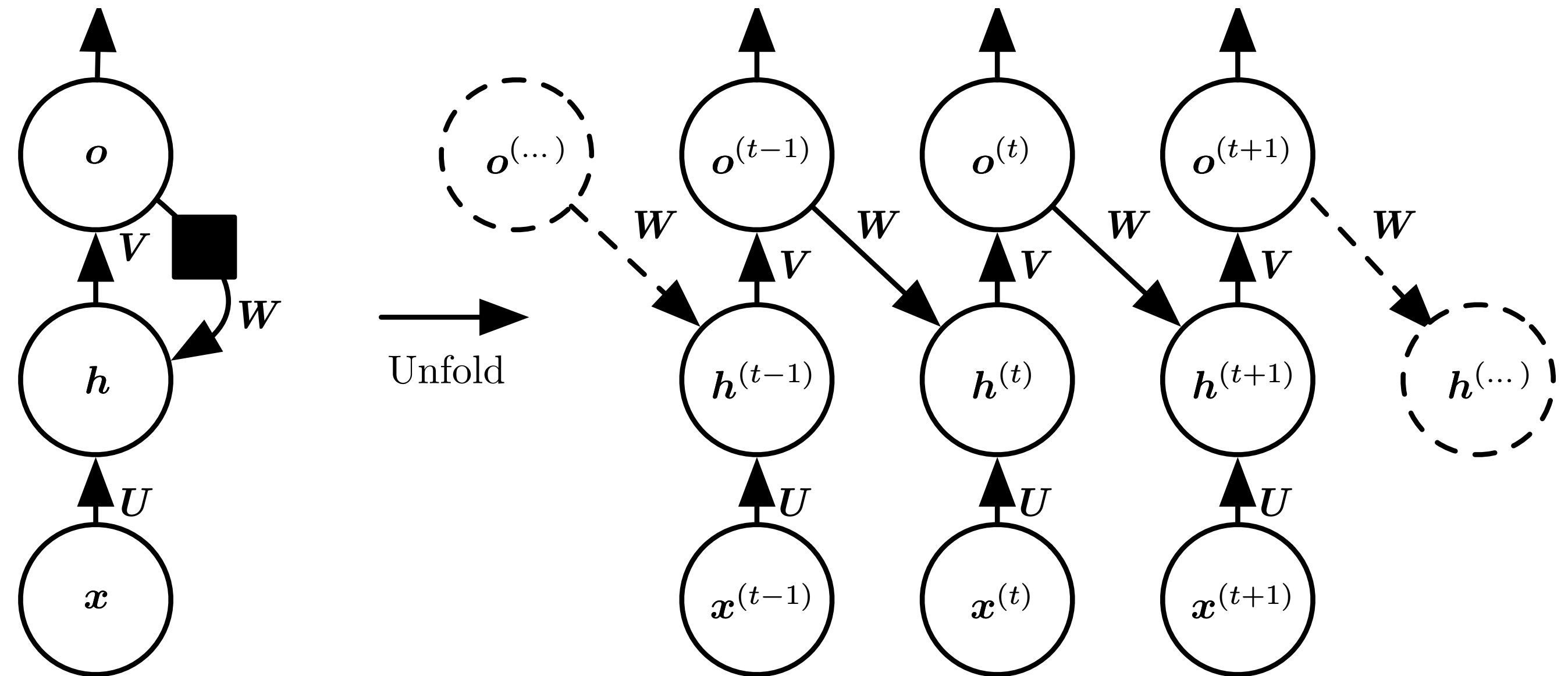


Questions

Alternative RNN architectures

RNN with Recurrent Output Units

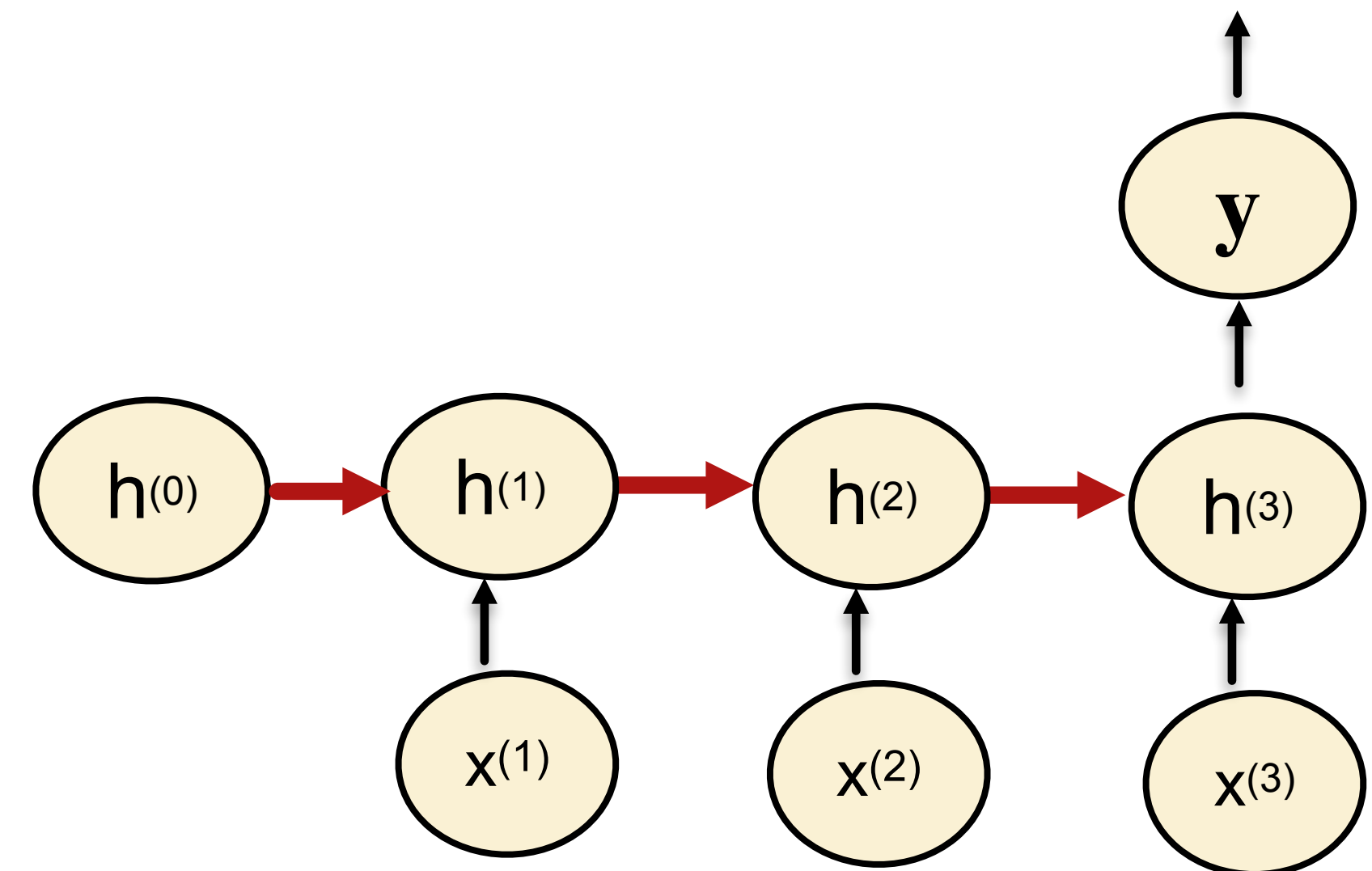
- Recurrent connection from prior output to current hidden unit
- This RNN is less “powerful” than the prior.
 - This network only retains output information
 - The prior network can choose to put any information into the hidden state and pass it to the future.



Alternative RNN architectures

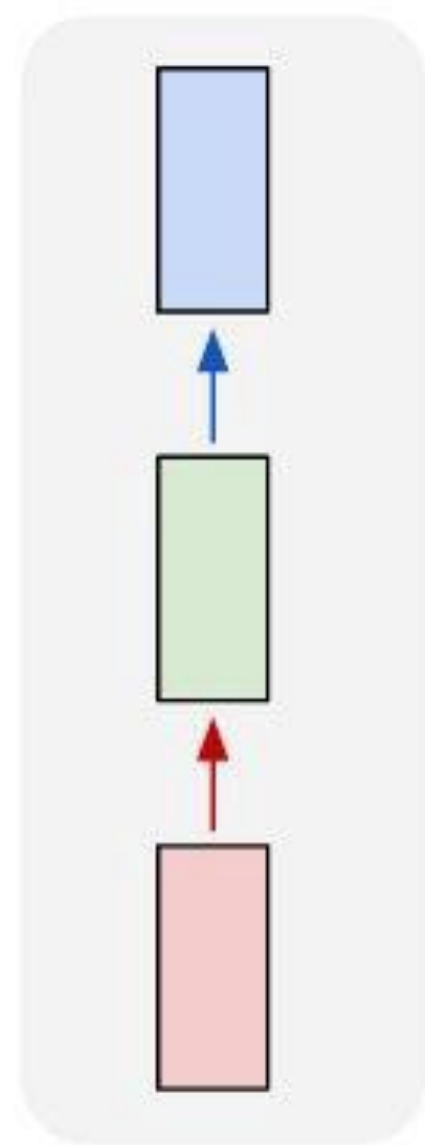
Sequential Input, Single Output

- Time unfolded RNN with a single output at the end of the sequence.
- This network is useful for summarizing a sequence and producing a fixed-size representation, which may be useful for further processing
- What application could this be useful for?

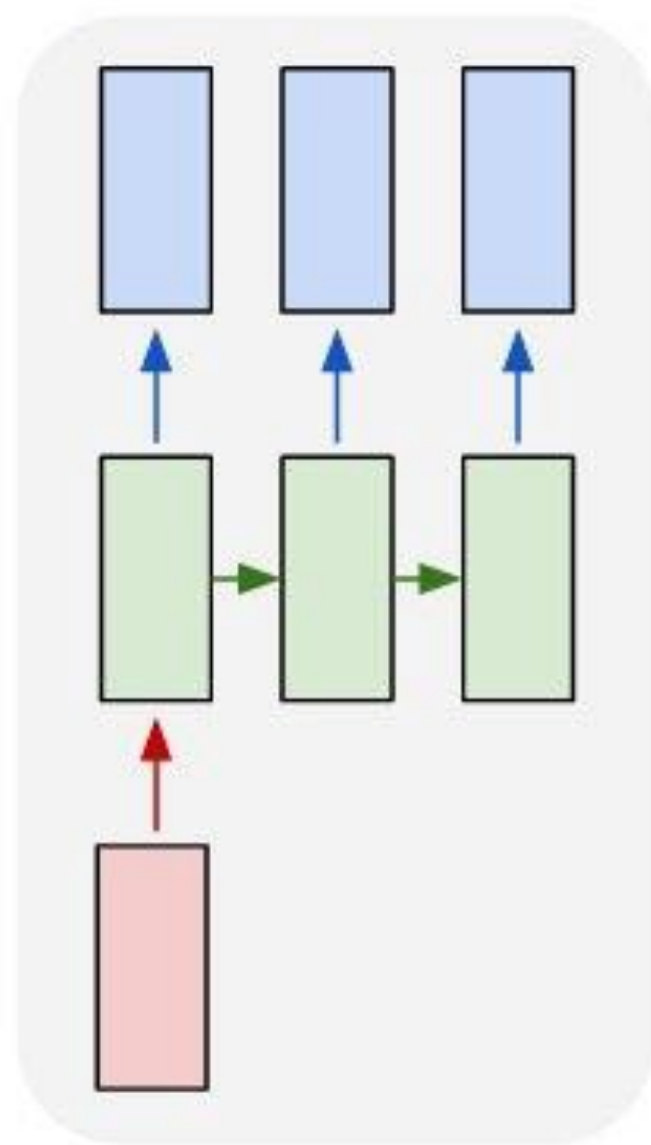


Other RNN Configurations

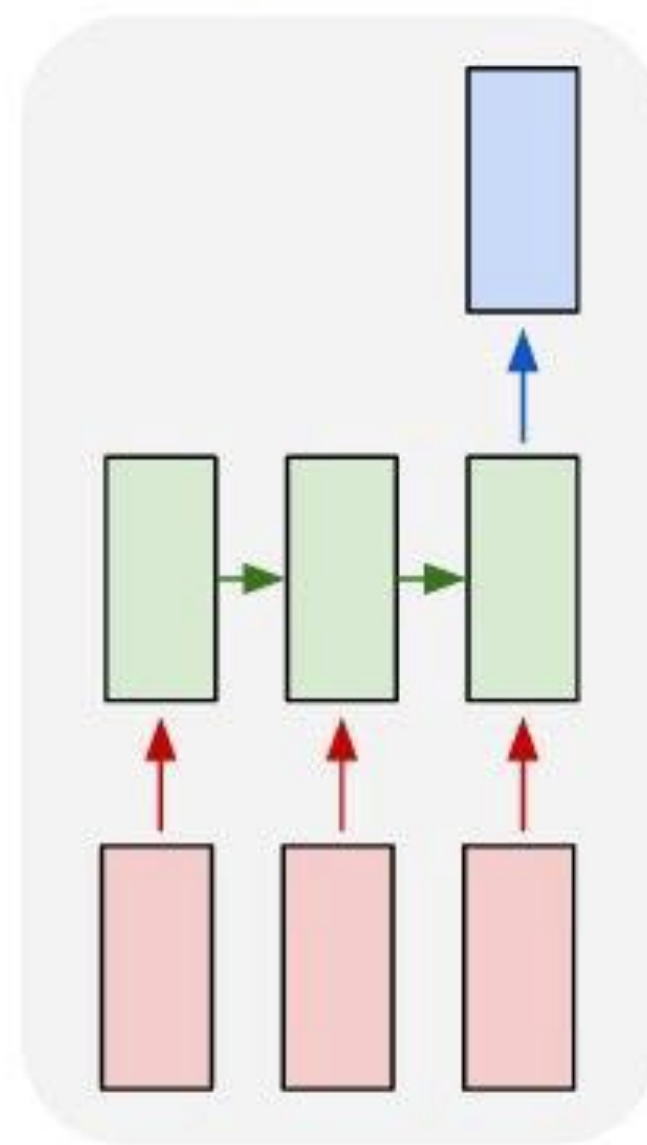
one to one



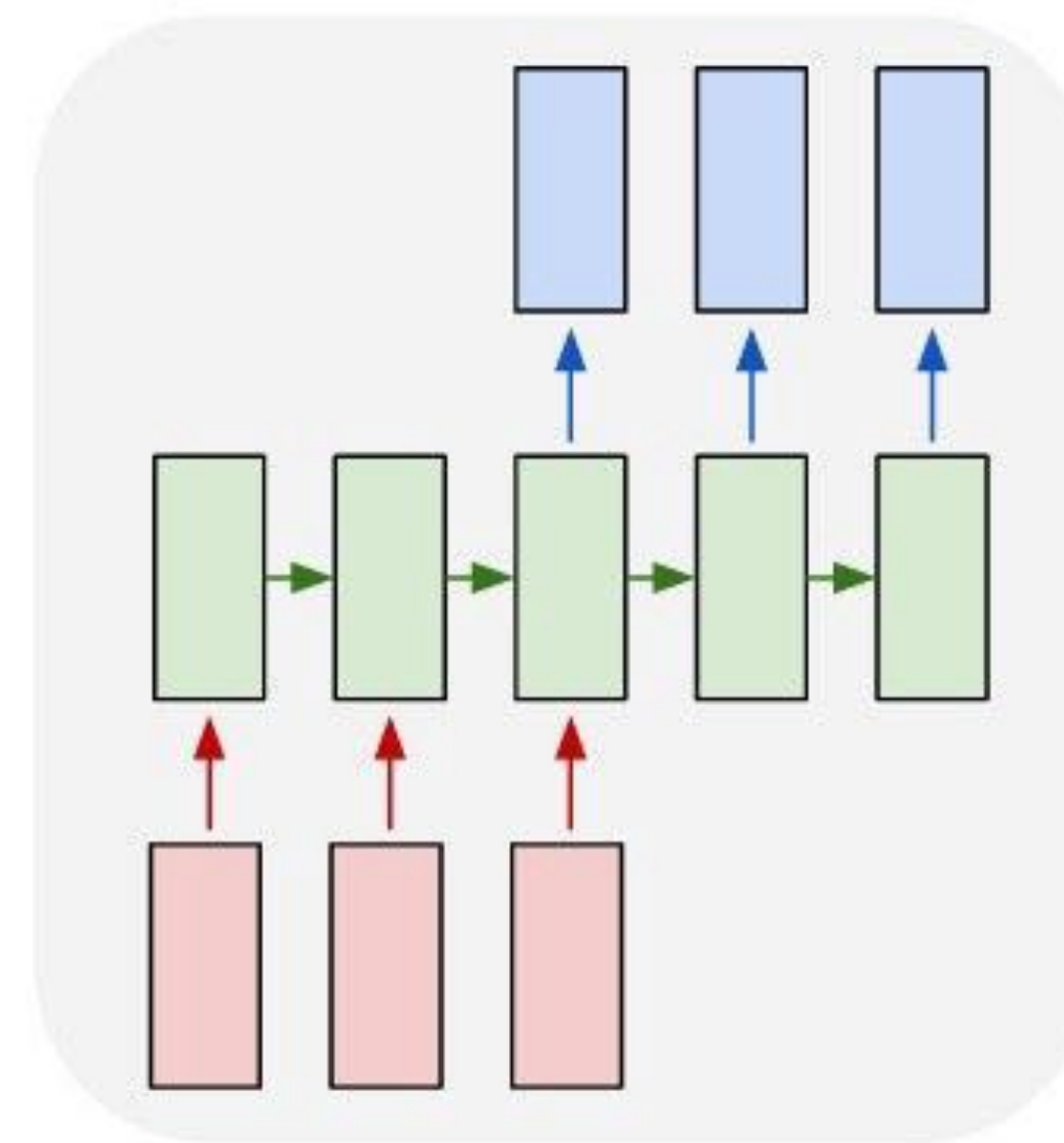
one to many



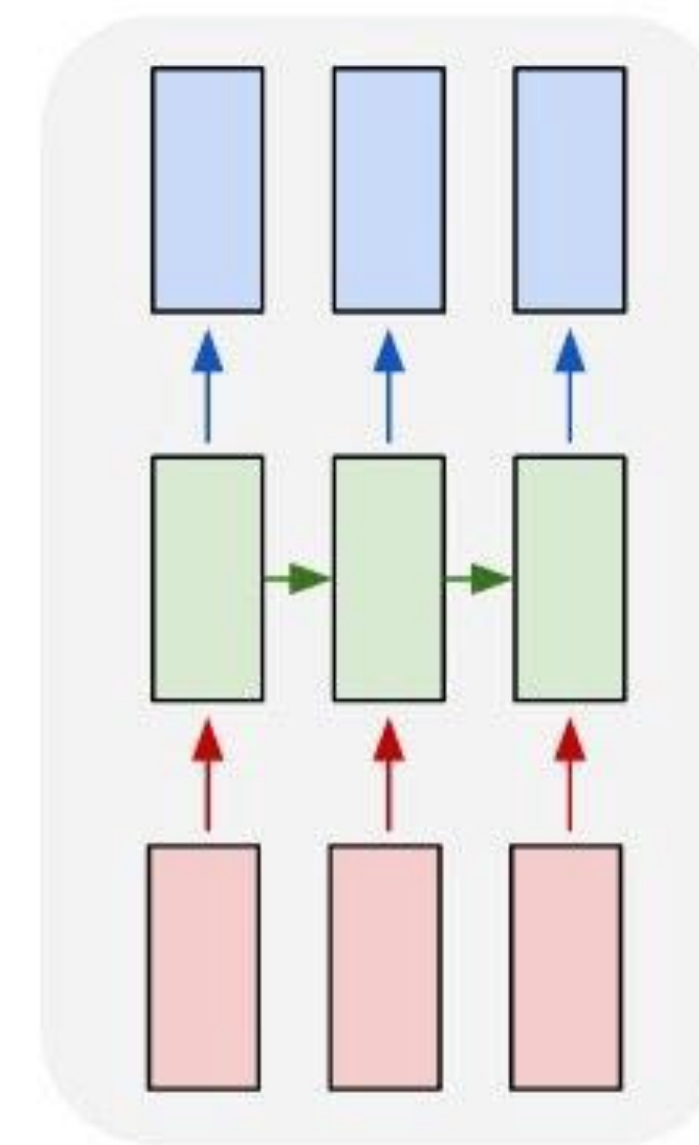
many to one



many to many



many to many



Summarizing RNN Training

- Essentially, an RNN cell is copied over time (unrolling or unfolding), with different inputs at different time steps
 - The weights are shared over time
- Key difference from traditional backpropagation is the use of shared parameters
 - Pretend that the weights are not shared and apply normal backpropagation to compute the gradients with respect to each copy of the weights
 - Summate (or average) the gradients over the various copies of each weight copy
 - Perform gradient descent update
- This algorithm is referred to as ***backpropagation through time (BPTT)***

Questions